Guia de teoria e preparação da defesa

Este guia parte do princípio de que sabes Java básico, mas ainda não dominas aplicações web. A ideia é perceberes o percurso de um pedido e conseguires localizar no código a responsabilidade de cada parte.

O essencial para começar

O AgroGest é um programa Java que corre num servidor chamado Tomcat. O utilizador abre páginas no navegador. Ao carregar num botão, o navegador envia um pedido HTTP. O Tomcat entrega o pedido ao código Java, que consulta ou altera os dados e devolve uma página HTML.

O programa tem duas áreas. O administrador gere produtos e a actividade agrícola. O cliente escolhe produtos, usa um carrinho e confirma compras. O sistema verifica as permissões e o stock no servidor.

Estuda nesta ordem

1. Browser, servidor e HTTP.
2. MVC: dados, controlador e vista.
3. Sessão, login e filtro.
4. Carrinho e compra com stock.
5. Concorrência: AtomicInteger e synchronized.
6. Gravação em ficheiro, WAR e Tomcat.

Uma frase para explicar o trabalho

“O AgroGest é uma aplicação web Java organizada em MVC. Usa Servlets para receber pedidos, JSP para apresentar páginas e mapas em memória para guardar os dados durante a execução. O login e o carrinho usam sessão, o filtro controla os acessos e a compra é sincronizada para não vender stock a mais. O estado é serializado para ficheiro e recuperado no arranque seguinte.”

Não precisas de decorar todos os ficheiros. Precisas de perceber esta explicação e conseguir demonstrar cada parte.

1. Browser, HTTP e Tomcat

O que é cliente e o que é servidor?

O **browser** é o cliente: mostra a interface e envia pedidos. O **Tomcat**, com o AgroGest, é o servidor: recebe pedidos, executa Java e prepara respostas. Podem estar no mesmo computador durante o desenvolvimento ou em computadores diferentes numa rede.

O browser não executa as classes Java do projecto. Recebe HTML, CSS e um pequeno JavaScript. O código Java e os dados do sistema ficam no servidor.

O que significa HTTP?

HTTP é o protocolo usado para pedidos e respostas na web. Um pedido indica uma rota, um método e, quando necessário, dados. A resposta traz um código de estado e conteúdo.

Exemplo	Significado no projecto
`GET /agrogest/catalogo`	Pedir a página do catálogo
`POST /agrogest/login`	Enviar email e palavra-passe
`POST /agrogest/compra`	Confirmar a compra do carrinho
HTTP 200	Pedido tratado com sucesso
HTTP 302	Redireccionamento para outra página
HTTP 400	Pedido com identificador ou dados inválidos
HTTP 403	Acesso sem permissão ou token de formulário inválido
HTTP 404	Recurso inexistente

GET serve para consultar. **POST** serve para operações que alteram estado, como criar, remover ou comprar. Usar POST não cifra os dados: a protecção do transporte em produção dependeria de HTTPS. Neste exercício local, o endereço usa HTTP.

O que faz o Tomcat?

O Tomcat é um contentor de Servlets. Gere as ligações HTTP, cria pedidos e respostas, mantém sessões e executa os Servlets. Também compila as JSP para gerar HTML. O projecto usa Tomcat 10.1 e `jakarta.servlet`, a API exigida pelo enunciado.

No endereço `http://localhost:8081/agrogest/catalogo`, `localhost` identifica o próprio computador, `8081` é a porta, `/agrogest` é o contexto da aplicação e `/catalogo` é a rota do Servlet.

Porque aparece isto em Sistemas Distribuídos?

Há comunicação entre cliente e servidor através de HTTP, com sessões, pedidos concorrentes e controlo de acesso. Este projecto usa uma instância do servidor. Não implementa replicação entre servidores nem uma base de dados distribuída.

2. MVC e organização do projecto

MVC separa três responsabilidades: **Model**, **View** e **Controller**. Isto permite alterar uma página sem reescrever o cálculo da compra.

Parte	O que faz	Ficheiros reais
Model	Representa entidades e regras do domínio	`Produto`, `Venda`, `VendaService`
View	Mostra os dados ao utilizador	`catalogo.jsp`, `factura.jsp`, CSS
Controller	Recebe o pedido e escolhe o que executar e mostrar	`CatalogoServlet`, `CompraServlet`

A camada Model é mais ampla do que a pasta `model`: inclui os dados, os serviços de negócio e o acesso ao repositório. A pasta `model` contém especificamente os POJOs.

O que é um POJO?

É uma classe Java simples. Por exemplo, `Produto` tem atributos privados, construtor e getters/setters. Representa um produto com nome, preço, quantidade e fornecedor. Não depende de Spring nem de um framework de base de dados.

`private BigDecimal preco;` guarda o preço. `getPreco()` permite lê-lo. A JSP pode escrever `${p.preco}`: a Expression Language usa o getter.

O que é um serviço?

Um serviço concentra regras. `GestaoService` impede remover uma campanha com cultivos. `VendaService` valida o carrinho, calcula a factura e desconta stock. O Servlet não deve repetir essas regras para cada página.

O que é o repositório?

`Estado` contém os mapas de produtos, clientes, campanhas e vendas. `DataStore` mantém esse estado e funciona como o monitor comum da aplicação. Todos os serviços recebem o mesmo DataStore criado no arranque.

Percurso de uma compra

1. O browser envia POST para `/compra`.
2. `AuthFilter` verifica sessão e token do formulário.
3. `CompraServlet` obtém o carrinho da sessão.
4. `VendaService.comprar` valida e altera o estado.
5. O Servlet limpa o carrinho e redirecciona para `/factura?id=...`.
6. `FacturaServlet` confirma quem pode ver a venda e abre `factura.jsp`.

Para a defesa: “O Servlet coordena o pedido. O serviço decide se a operação pode acontecer. A JSP mostra o resultado.”

3. Servlet, JSP, JSTL e Expression Language

Servlet

Um Servlet é uma classe Java que recebe pedidos HTTP no servidor. A anotação `@WebServlet("/compra")` associa a classe à rota. `doGet` trata consultas e `doPost` trata os formulários que enviam alterações.

Código	O que significa
`req.getParameter("email")`	Lê o campo email enviado pelo browser
`req.setAttribute("produtos", lista)`	Disponibiliza uma lista para a JSP deste pedido
`req.getSession()`	Obtém a sessão associada ao browser
`resp.sendRedirect(...)`	Manda o browser fazer um novo pedido para outra rota
`getRequestDispatcher(...).forward(...)`	Passa o pedido actual para uma JSP no servidor

Forward aproveita os atributos do mesmo pedido. **Redirect** cria outro pedido e altera o endereço no browser. Depois de um POST bem sucedido, o projecto faz redirect. Assim, actualizar a página da factura não volta a enviar o formulário de compra.

Este padrão chama-se **Post/Redirect/Get**. A compra também sincroniza a sessão e esvazia o carrinho após o sucesso, para que um segundo envio na mesma sessão não crie outra venda com os mesmos itens.

JSP

Uma JSP é uma página do lado do servidor que mistura HTML com tags para apresentar dados. O Tomcat transforma-a numa implementação executável e devolve o HTML gerado. O browser não recebe o código JSP.

As JSP ficam em `WEB-INF/views`. O utilizador abre `/catalogo`, cujo Servlet escolhe a JSP. Não precisa de conhecer o caminho físico do ficheiro.

JSTL e Expression Language

JSTL fornece tags prontas. `c:forEach` percorre os produtos. `c:if` mostra algo quando uma condição é verdadeira. `c:out` escreve texto escapando caracteres de HTML.

Expression Language, ou **EL**, é a escrita `${...}`. Por exemplo, `${p.nome}` lê o nome de um produto. `${empty linhas}` verifica se a lista está vazia.

O projecto usa tags próprias pequenas, como `layout.tag`, para repetir o menu e o cabeçalho. Isto evita copiar a mesma interface para todas as páginas.

O HTML `required` ajuda o utilizador a preencher. O servidor volta a validar tudo, porque um pedido HTTP pode ser enviado sem usar o formulário do browser.

4. Sessão, autenticação e autorização

Porque precisamos de sessão?

HTTP não associa automaticamente um pedido ao anterior. Sem um mecanismo de sessão, o servidor não saberia que o pedido do carrinho pertence à pessoa que acabou de entrar.

O Tomcat cria uma **HttpSession** e associa-lhe um identificador. Normalmente envia esse identificador num cookie chamado **JSESSIONID**. O browser devolve o cookie nos pedidos seguintes. Os dados da sessão ficam no servidor.

No AgroGest, a sessão guarda o ID do utilizador, o carrinho e o token dos formulários. Não guarda a palavra-passe original. A configuração faz a sessão expirar após 30 minutos sem actividade.

Três locais diferentes para dados

Âmbito	Duração e utilização
Pedido (`request`)	Só durante aquele pedido; exemplo: produtos que a JSP vai mostrar
Sessão (`session`)	Entre pedidos daquele browser; exemplo: carrinho
Aplicação (`ServletContext`)	Partilhado por todos os utilizadores daquela aplicação; exemplo: DataStore

O stock deve estar na aplicação porque todos compram do mesmo stock. O carrinho deve estar na sessão porque cada cliente tem a sua selecção.

Autenticação e autorização

Autenticação responde a “quem és?”. O email e a palavra-passe identificam a conta. **Autorização** responde a “o que podes fazer?”. O perfil determina se a conta pode entrar na área administrativa.

O `AuthFilter`, marcado com `@WebFilter("/*")`, executa-se antes dos controladores. As rotas do cliente exigem sessão. As rotas `/admin/*` devolvem 403 a quem não for administrador, mesmo que a pessoa escreva o endereço directamente.

Esconder um botão não é controlo de acesso. A verificação tem de acontecer no servidor. As facturas ainda têm outra regra: `VendaService.factura` verifica o dono da venda. Um cliente autenticado continua sem permissão para abrir a factura de outro.

Ao entrar, o sistema muda o ID de sessão. Ao sair, invalida a sessão. Dois separadores do mesmo browser normalmente partilham a mesma sessão; para demonstrar dois clientes em simultâneo, usa uma janela privada ou outro navegador.

5. Carrinho e factura

Como é representado o carrinho?

É um `Map<Integer, BigDecimal>` na sessão. A chave é o ID do produto. O valor é a quantidade escolhida. Exemplo: produto 1 com 3 kg e produto 2 com 1 saco.

Um Map permite procurar directamente a quantidade pelo ID. Adicionar o mesmo produto soma a quantidade existente. Actualizar substitui-a. Remover elimina a entrada. Esvaziar usa `clear()`.

O carrinho não é uma venda e não reserva stock. Antes de confirmar, o servidor consulta novamente os produtos reais, os seus preços, a validade e o stock.

Cálculos da factura

Exemplo: 3 kg a 85,00 MT/kg e 1 saco a 1850,00 MT.

Cálculo	Resultado
Linha 1: 3 × 85,00	255,00 MT
Linha 2: 1 × 1850,00	1850,00 MT
Subtotal: 255,00 + 1850,00	2105,00 MT
IVA: 2105,00 × 0,16	336,80 MT
Total: 2105,00 + 336,80	2441,80 MT

O enunciado exige IVA de 16%, por isso o código usa a constante `TAXA_IVA`. Esta explicação aplica a regra do exercício, sem tentar determinar legislação fiscal.

Porque usar BigDecimal?

`double` usa representação binária e pode introduzir pequenas diferenças em valores decimais. `BigDecimal` permite trabalhar com preços e quantidades decimais e controlar o arredondamento. O projecto arredonda cada linha para 2 casas, soma as linhas e calcula o IVA com `HALF_UP`.

Porque guardar dados repetidos na venda?

`ItemVenda` guarda nome do produto, unidade, preço, quantidade e subtotal do momento da compra. `Venda` guarda uma cópia dos dados do cliente. Se o preço do catálogo mudar ou o produto for removido, a factura antiga deve continuar igual.

O número `FT-ANO-ID` usa o ID sequencial da venda. O contador é gravado juntamente com os dados, por isso não volta a 1 a cada reinício.

6. Concorrência e compra “tudo ou nada”

O que é concorrência?

Vários pedidos podem ser tratados ao mesmo tempo por threads do Tomcat. Uma thread pode estar a confirmar uma compra enquanto outra confirma outra compra ou o administrador edita stock.

Imagina que existe 1 unidade. Sem protecção, dois clientes podem ler “há 1”, ambos vender e deixar o stock negativo. Isto é uma **condição de corrida**: o resultado depende da ordem imprevista das operações.

O que faz synchronized?

`synchronized (store)` usa o objecto DataStore como monitor. Enquanto uma thread executa esse bloco, outra thread que queira entrar num bloco sincronizado sobre o mesmo objecto espera. Quando a primeira sai, a próxima pode reavaliar os dados já actualizados.

Todos os serviços que escrevem no estado partilham o mesmo monitor. Sincronizar apenas a venda não chegaria se a reposição ou a edição de stock pudesse alterar o produto ao mesmo tempo sem esse lock.

Como acontece a atomicidade da venda?

O serviço faz o seguinte dentro do mesmo bloco sincronizado:

1. Confirma que o comprador é cliente e que o carrinho não está vazio.
2. Valida todos os itens: existência, quantidade, validade e stock.
3. Calcula os itens e a factura sem alterar o stock.
4. Só depois de todas as validações passarem, desconta todos os produtos e regista a venda.

Se o último produto falhar, os primeiros ainda não foram descontados. **Atomicidade** significa que a operação lógica acontece por completo ou não acontece. Esta implementação evita alterações parciais perante falhas de validação esperadas. Não promete resistir a uma falha fatal da JVM no meio das instruções.

AtomicInteger faz a mesma coisa?

Não. `AtomicInteger.incrementAndGet()` gera um novo número com segurança entre threads. Resolve a geração de IDs. Não valida stock, não torna duas listas consistentes e não transforma uma compra inteira numa operação atómica.

Os testes colocam dois pedidos em competição pela última unidade. Só um consegue comprar e o stock acaba em zero. Outro teste força falta de stock no segundo produto e confirma que o primeiro fica intacto.

Limite: estes locks protegem uma aplicação numa JVM. Dois servidores separados teriam memórias e locks diferentes e exigiriam coordenação adicional.

7. Dados em memória e serialização

Onde estão os dados enquanto o programa corre?

Estão em mapas `LinkedHashMap`, guardados em `Estado`. A chave é um ID e o valor é um objecto Java. As consultas devolvem listas para ordenar e mostrar nas páginas. A memória é rápida, mas não conserva dados depois de o processo terminar.

O enunciado proíbe base de dados nesta versão. Portanto, não existem tabelas SQL, JDBC ou MySQL no projecto.

O que significa Serializable?

`Serializable` identifica objectos que podem ser representados num fluxo de bytes por serialização Java. `ObjectOutputStream` escreve o estado. `ObjectInputStream` reconstrói os objectos a partir do ficheiro.

O ficheiro `.ser` contém o conjunto do domínio: utilizadores, fornecedores, produtos, campanhas, cultivos, colheitas, vendas e contadores. O `serialVersionUID` identifica a versão de serialização de uma classe. Alterar classes de forma incompatível pode exigir migração do estado; não se deve prometer que todo o ficheiro antigo abre após qualquer mudança de código.

Quem grava e carrega?

`AplicacaoListener` implementa `ServletContextListener` e tem `@WebListener`. O Tomcat chama `contextInitialized` ao iniciar o contexto e `contextDestroyed` ao encerrá-lo normalmente. O listener carrega o estado no arranque e solicita a gravação no encerramento.

O administrador também pode gravar manualmente em **Guardar dados**. A leitura usa um filtro de tipos para limitar os objectos aceites do ficheiro local.

Porque usar um temporário e ATOMIC_MOVE?

Escrever directamente sobre o ficheiro principal pode deixá-lo a meio se houver uma falha. O serviço escreve primeiro noutro ficheiro na mesma pasta. Depois de terminar a escrita, substitui o destino com uma operação atómica. Mantém ainda um backup da gravação anterior válida.

Se o principal estiver danificado, tenta o backup. Se os dois estiverem inválidos, o arranque falha sem apagar os ficheiros. Uma gravação falhada não deve ser apresentada como concluída.

Atomicidade e durabilidade são coisas diferentes

A compra é atómica em memória perante erros de validação. **Durabilidade** significa conservar o resultado depois de o processo terminar. Neste projecto, depende da gravação em ficheiro. Um corte de energia antes de guardar pode perder compras feitas depois da última gravação.

Guardar um ficheiro não equivale a implementar uma base de dados transaccional completa. Esta distinção é uma boa resposta para a defesa.

8. Segurança, validação e relatórios

Hash da palavra-passe

O sistema não guarda a palavra-passe original. Usa PBKDF2 com HMAC-SHA256, salt aleatório e 210 000 iterações. O salt faz com que duas contas com a mesma palavra-passe tenham valores guardados diferentes.

Ao entrar, o programa deriva o hash da palavra-passe recebida usando os parâmetros guardados e compara o resultado. Um hash não é uma cifra para posteriormente "desencriptar" a palavra-passe.

Os exemplos de credenciais no README são para utilizar as contas de demonstração. O estado serializado guarda hashes, não essas palavras-passe originais.

Token dos formulários e texto escapado

O campo oculto `csrf` transporta um token associado à sessão. O filtro exige esse token nos pedidos POST. É uma protecção contra pedidos de alteração enviados por uma página externa sem o formulário legítimo.

`c:out` escapa texto fornecido pelo utilizador. Por exemplo, um nome com `<script>` deve aparecer como texto, sem executar código no browser. Isto reduz o risco de **XSS**, que é a injecção de código numa página.

Autenticação, autorização, CSRF e XSS são problemas diferentes. Ter login não resolve os outros automaticamente.

Validações de negócio

Situação	Resposta do servidor
Email já usado	Rejeita o novo registo
Quantidade negativa ou zero na compra	Rejeita o item
Stock insuficiente	Rejeita toda a compra
Campanha com cultivos	Bloqueia a remoção
Cultivo com colheitas	Bloqueia a remoção
Conta de cliente numa rota admin	HTTP 403
Cliente numa factura alheia	HTTP 403

Relatórios

O número de compras conta facturas do cliente. O volume soma os totais em MT com IVA. Os relatórios ordenam por nome, volume ou frequência. "Sem compras" filtra quem tem zero facturas. A ordenação usa comparadores Java sobre listas obtidas do repositório.

O valor da colheita é quantidade em kg × preço por kg. É um registo de produção, separado do volume comercial das facturas.

9. Maven, WAR, deploy e systemd

JDK, Java e Maven

O Java Runtime executa programas Java. O **JDK** inclui ferramentas de desenvolvimento, como `javac`, para compilar. O **Maven** organiza a compilação, descarrega dependências, executa testes e empacota o projecto de acordo com `pom.xml`.

O código usa Java 17. A dependência `jakarta.servlet-api` tem âmbito `provided`: é necessária para compilar, mas o próprio Tomcat fornece a implementação em execução. JSTL acompanha o WAR porque o contentor não a fornece automaticamente como parte da aplicação.

O que é um WAR?

WAR significa **Web Application Archive**. É um pacote de aplicação web. Contém as classes compiladas, JSP, CSS, bibliotecas e configuração. O Maven gera `target/agrogest.war`.

Deploy significa colocar essa aplicação no servidor para poder receber pedidos. No Tomcat, copiar `agrogest.war` para `webapps` permite publicar o contexto `/agrogest`.

O ZIP inclui um WAR já compilado e um Tomcat portátil. Por isso, para experimentar, executas `iniciar.sh` ou `iniciar.bat` com Java instalado. Para alterar e recompilar o código, precisas também de JDK e Maven.

Portas e configuração

O Connector do `server.xml` define a porta HTTP. A versão incluída usa 8081. A porta 8105 serve para a paragem local do Tomcat. Se uma porta estiver ocupada, é necessário alterá-la ou parar o processo que a usa.

O endereço 127.0.0.1 aceita ligações do próprio computador. Para uma demonstração entre máquinas, o Connector precisa de aceitar ligações na interface de rede e a firewall precisa de permitir a porta. O README explica os passos.

O que é systemd?

É um gestor de serviços em muitos sistemas Linux. Um ficheiro `.service` define como iniciar, parar e acompanhar o Tomcat sem depender de um terminal aberto. O projecto inclui `deploy/agrogest.service`.

Comando	Função
`systemctl start agrogest`	Iniciar o serviço
`systemctl stop agrogest`	Parar normalmente
`systemctl restart agrogest`	Parar e iniciar
`systemctl status agrogest`	Consultar o estado
`journalctl -u agrogest`	Consultar os logs do serviço

A instalação do serviço depende da máquina de destino. O funcionamento do programa foi testado no Tomcat em Linux; o serviço não foi activado num sistema com systemd nesta execução.

10. Onde estudar no código

Ordem de leitura dos ficheiros Java

1. `model/Produto.java`: reconhecer atributos, construtor e getters/setters.
2. `repo/Estado.java` e `DataStore.java`: localizar os mapas e contadores.
3. `servlet/CatalogoServlet.java`: ver o percurso GET, lista, atributo e JSP.
4. `filter/AuthFilter.java`: localizar sessão, perfil e verificação do token.
5. `servlet/CarrinhoServlet.java`: perceber o Map da sessão.
6. `servlet/CompraServlet.java` e `service/VendaService.java`: seguir a compra até ao desconto do stock.
7. `service/PersistenciaService.java` e `listener/AplicacaoListener.java`: seguir guardar e restaurar.

Os caminhos acima são relativos a `src/main/java/mz/agrogest/`. As vistas ficam em `src/main/webapp/WEB-INF/views/`.

Preparação recomendada

Primeiro, executa a aplicação e faz uma compra. Depois, acompanha no código os seis passos da secção MVC. Por fim, ensaia o roteiro em APRESENTACAO_E_DEMONSTRACAO.md e demonstra uma regra bloqueada, como remover uma campanha com cultivos. Isso mostra compreensão do comportamento, além da interface.

Referências para confirmar os conceitos

Enunciado AgroGest, 7 páginas; documentação de Jakarta Servlet 6.0; documentação oficial de Tomcat 10.1; Maven WAR Plugin; especificação de serialização Java 17. Os links completos estão no README. As regras específicas de stock, perfis e IVA seguem o enunciado.

11. Perguntas prováveis da defesa

Porquê MVC? Para separar a apresentação, a recepção dos pedidos e as regras de negócio. Assim, uma mudança de interface não exige mudar o cálculo da venda.

Onde fica o carrinho? Na HttpSession do cliente. O stock fica no DataStore partilhado da aplicação.

Como evitas vender mais do que existe? Todas as validações e o desconto dos itens ficam dentro do mesmo `synchronized(store)`. Cada compra valida o stock actualizado.

E se faltar stock no segundo item? A validação acontece antes de alterar qualquer item. A compra é rejeitada sem descontar o primeiro.

Porquê AtomicInteger e synchronized? AtomicInteger protege o incremento dos IDs. synchronized protege a operação com vários produtos e mapas.

Como proteges as facturas? Verifico no servidor se o utilizador é o dono da venda ou administrador. Caso contrário, devolvo 403.

O que acontece se reiniciar? O listener tenta carregar o ficheiro guardado. O carrinho da sessão não faz parte desse estado. Os clientes, produtos, vendas e contadores são recuperados.

E se faltar energia antes de guardar? Recupero a última gravação válida. Alterações posteriores podem perder-se.

Como sabes que funciona? Há testes JUnit para as regras de negócio e um teste HTTP que inicia o Tomcat, usa os formulários e verifica as páginas e o reinício. Consulta os resultados em VALIDACAO.md.

O sistema distribui dados entre servidores? Não. É uma aplicação cliente-servidor com comunicação HTTP e uma instância de memória. A coordenação entre vários servidores seria trabalho adicional.

12. Conceitos acrescentados na revisão

Um formulário também pode ficar antigo

Imagina que o administrador abre um produto com 20 unidades. Enquanto o formulário está aberto, um cliente compra 3 unidades. O stock correcto passa a 17. Se o formulário antigo voltasse a enviar 20 e o servidor aceitasse sem conferir, as 3 unidades vendidas reapareceriam no stock.

O `synchronized` protege cada operação enquanto ela executa, mas não mantém um bloqueio durante o tempo em que a pessoa está a preencher um formulário.

Controlo optimista de versão

Cada produto tem um número de versão. Uma compra, uma reposição ou uma edição aumenta esse número. O formulário envia a versão que existia quando foi aberto.

Ao guardar, o serviço compara essa versão com a versão actual, dentro do mesmo `synchronized(store)`. Se forem diferentes, recusa a edição antiga e pede para reabrir o produto. Se forem iguais, pode guardar e incrementar a versão.

Este mecanismo chama-se controlo optimista: deixa a pessoa editar sem bloquear os outros, mas verifica no fim se os dados entretanto mudaram. O campo fica oculto no formulário; a validação importante continua a acontecer no serviço.

Cópias para consulta

As consultas de produtos devolvem cópias dos objectos, incluindo o stock e a versão lidos no mesmo bloco sincronizado. Assim, a página recebe um conjunto estável de valores e não consegue alterar o objecto interno por fora do serviço.

Bloqueio da pasta de dados

Dois processos Java têm memórias diferentes. O `synchronized` de um processo não bloqueia o outro. Para evitar que dois AgroGest gravem na mesma pasta, `TravaDados` adquire um `FileLock` num ficheiro local durante a vida da aplicação.

O bloqueio é libertado no encerramento e pelo sistema operativo se o processo terminar abruptamente. O ficheiro vazio pode continuar a existir. Isso não significa que a pasta continue bloqueada. Esta medida impede o uso simultâneo da mesma pasta; não implementa replicação ou partilha de stock entre servidores.

Como explicar na defesa

“A revisão protege também formulários que ficaram abertos enquanto os dados mudavam. Conferimos a versão do produto antes de guardar, para não desfazer uma compra ou reposição. Para o ficheiro de estado, impedimos duas instâncias de usar a mesma pasta. Mantivemos o MVC, os mapas em memória e a serialização pedidos no enunciado.”